Corrected

AARHUS UNIVERSITET

Prof. Akhil Arora

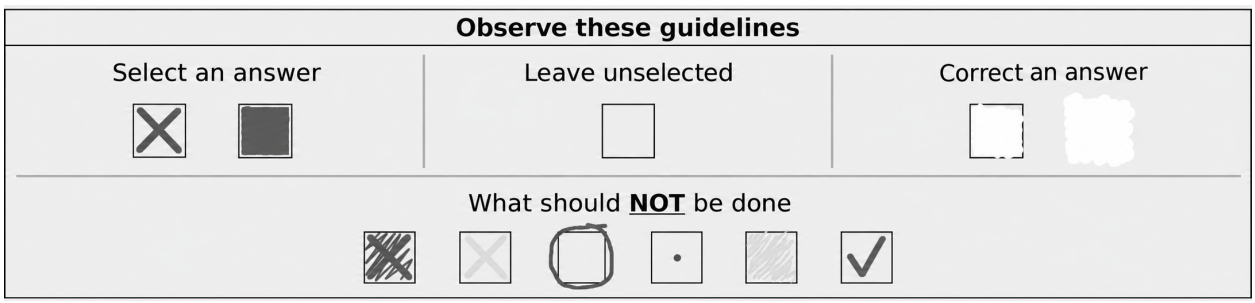
Natural Language Processing (NLP) – 520242U008
Midterm preparation – 2026
Suggested practice time: 42 minutes

Mock Exam 2026 Solution

Student ID : _____ Signature :

Do not turn the page before the start of the exam. This booklet is intended for double-sided printing and has 11 pages, the last ones possibly blank. Do not unstaple.

- This is a closed book exam. Non-programmable calculators are allowed. No other electronic devices of any kind are allowed.
- Place on your desk: your student ID, writing utensils, one double-sided A4 page cheat sheet if you have one; place all other personal items below your desk.
- This mock exam has **14 questions**. The suggested practice time is **42 minutes**.
- This exam has **multiple-choice** questions of varying difficulty. Each question is worth **one point**.
- Each question has **exactly one** correct answer. For each question, mark the box corresponding to the correct answer.
- An **answer sheet is included at the end of this booklet**. Record **all your answers on that sheet**. Only answers marked on the answer sheet will be graded. You can use the blank space in this booklet as scrap paper.
- Use a **black or dark blue ballpen** and clearly erase with **correction fluid** if necessary.
- If a question turns out to be wrong or ambiguous, we may decide to nullify it.
- **Do not tick** the boxes: **cross** them or **fill them in completely**. Do not write close to the boxes, and make sure an erased box is left **completely white**.



CORRECTED

Question 1

What is the primary purpose of backpropagation in neural networks?

A. To compute gradients of the loss function with respect to the input data.

B. To adjust the learning rate dynamically during training.

C. To directly update the weights while propagating error values backward through the network.

D. To compute gradients of the loss function with respect to the network's parameters.

Answer: D *Difficulty: Easy; Topic: Neural network basics / backpropagation*

Backpropagation applies the chain rule to compute the loss gradients with respect to the trainable parameters. An optimizer uses those gradients to update the parameters. Gradients with respect to inputs can also be computed, but this is not the primary purpose. Backpropagation itself does not choose the learning-rate schedule or perform the optimizer's parameter update.

Question 2

A model has a perplexity of 20 on dataset A and 40 on dataset B, using the same tokenizer and evaluation procedure. Which conclusion is justified by these values alone?

A. The model performs exactly twice as well on every downstream task on A as on B.

B. The model assigns exactly twice as much probability to every token in A as to every token in B.

C. The model's vocabulary coverage is higher on B than on A.

D. The model has a lower average negative log-likelihood per token on A, but this alone does not establish that A is easier or that the model is twice as good on A.

Answer: D *Difficulty: Medium; Topic: Evaluation / perplexity*

Perplexity is the exponential of the average negative log-likelihood per token. With the same evaluation conventions, 20 versus 40 means the average negative log-likelihood is lower on A by $\ln 2$ nats per token. These values do not determine per-token probability ratios, vocabulary coverage, or downstream performance, and the datasets may differ in distribution or difficulty. Perplexities can be reported across datasets; the interpretation must account for those differences.

Question 3

What is a key reason to use PPO (Proximal Policy Optimization), rather than an unconstrained policy-gradient update such as basic REINFORCE, in an RLHF pipeline?

A. PPO guarantees that the model never forgets previously learned tasks.

B. PPO guarantees that exploration never produces an undesirable token.

C. PPO makes the learned reward function exact and removes all reward-estimation error.

D. PPO helps stabilize updates by discouraging excessively large changes to the policy.

Answer: D *Difficulty: Medium; Topic: RLHF / PPO*

PPO's clipped surrogate objective limits the incentive for excessively large changes in action-probability ratios relative to the rollout policy. This improves update stability. Clipping is not a guarantee against catastrophic forgetting, undesirable generation, or reward-model error. In RLHF, a separate KL penalty relative to a reference model may also be used.

CORRECTED

Question 4

Consider the following statements about standard Word2Vec (without subword features).

1. Word2Vec can produce a distinct learned embedding for a word that never appeared in its training vocabulary, without any additional mechanism.
2. Standard CBOW and Skip-gram do not explicitly encode the relative positions of words within the context window.
3. Shuffling all words within each sentence leaves the training co-occurrences, and therefore the learned representations, unchanged.

Which of the above statements are true?

A. 2 only.

B. 1 and 3.

C. 1 and 2.

D. 2 and 3.

Answer: A *Difficulty: Hard; Topic: Word embeddings / Word2Vec*

Only statement 2 is true. Standard Word2Vec learns embeddings for words in its training vocabulary and has no built-in mechanism for deriving an embedding for a truly unseen word. Its usual CBOW and Skip-gram objectives do not encode each context word's relative position. Shuffling a sentence can change which words fall within a local context window, changing the training pairs. Ignoring order within a fixed window is different from ignoring changes to window membership.

Question 5

Which of the following describes a key limitation of a standard RNN when its current hidden state is used to summarize a long sequence?

A. It must compress the preceding context into a fixed-size hidden state, which can make retaining all relevant information difficult.

B. Its computational complexity is always quadratic in sequence length, even when the hidden dimension is fixed.

C. It uses a different set of trainable parameters at each timestep.

D. It cannot learn nonlinear representations because its hidden-state updates must be linear.

Answer: A *Difficulty: Easy; Topic: RNN limitations*

The current hidden state is a fixed-size summary of the past, which can create an information bottleneck for long contexts. With a fixed architecture and hidden size, a standard RNN performs a linear number of recurrent steps in sequence length. Its parameters are shared across timesteps, and its usual state update includes a nonlinear activation.

Question 6

In a Transformer encoder-decoder model, each decoder block uses masked self-attention followed by encoder-decoder cross-attention. What is the role of the causal mask in decoder self-attention?

A. It enables the decoder to attend to all encoder outputs, aligning the source and target sequences.

B. It randomly removes some encoder outputs as a regularization mechanism.

C. It allows every decoder position to attend to all future target positions during training.

D. It prevents a decoder position from attending to future target positions, preserving the autoregressive prediction constraint.

Answer: D *Difficulty: Easy; Topic: Transformer decoder / masked self-attention*

The causal mask blocks attention to future target positions. With the usual shifted decoder inputs, a prediction cannot access the target token it is supposed to predict. Attending to encoder outputs is the role of cross-attention. The causal mask follows a fixed positional rule rather than randomly removing positions.

Question 7

A training corpus contains the following six sentences. Split words by whitespace; do not add sentence-boundary or other special tokens, and do not form bigrams across sentence boundaries.

```
the cat sat on the mat
the dog chased after the cat
a cat and a dog ran
she sat on the sofa
they bought a new mat
the sofa was comfortable
```

There are 18 unique tokens and 32 tokens in total. To assign non-zero probability to unseen bigrams over this fixed vocabulary, use add-one Laplace smoothing:

$$P(A \mid B) = \frac{\text{count}(B, A) + \alpha}{\text{count}(B) + \alpha\,|V|}, \qquad \alpha = 1.$$

What are $P(\text{sat} \mid \text{on})$ and $P(\text{on} \mid \text{sat})$?

A. None of the other options is correct.

B. $P(\text{sat} \mid \text{on}) = \frac{1}{20}$ and $P(\text{on} \mid \text{sat}) = \frac{3}{20}$.

C. $P(\text{sat} \mid \text{on}) = 1$ and $P(\text{on} \mid \text{sat}) = 0$.

D. $P(\text{sat} \mid \text{on}) = \frac{3}{20}$ and $P(\text{on} \mid \text{sat}) = \frac{1}{20}$.

Answer: B *Difficulty: Hard; Topic: N-gram LM / Laplace smoothing*

The bigram `sat on` occurs twice, whereas `on sat` never occurs. Both `sat` and `on` occur twice, so both denominators are $2 + 18 = 20$. Thus $P(\text{sat} \mid \text{on}) = (0 + 1)/20 = 1/20$, and $P(\text{on} \mid \text{sat}) = (2 + 1)/20 = 3/20$. Smoothing unseen bigrams over a fixed vocabulary does not itself create representations for out-of-vocabulary words.

CORRECTED

Question 8

In a Transformer's scaled dot-product self-attention layer, which statement is correct?

A. Self-attention always permits attention only to previous positions, even without an attention mask.

B. The attention weights are obtained by applying softmax directly to the query matrix, across its feature dimensions.

C. Increasing the query/key dimension d_k does not change the arithmetic cost of computing QK^T for a fixed sequence length.

D. Dividing the query-key dot products by $\sqrt{d_k}$ helps control their scale and reduces the risk of softmax saturation when d_k is large.

Answer: D *Difficulty: Medium; Topic: Self-attention mechanics*

Scaled dot-product attention computes $\text{softmax}(QK^\mathsf{T}/\sqrt{d_k})V$, with softmax over the key positions for each query. Under the usual variance assumptions, dot-product variance grows with d_k; scaling helps avoid extreme softmax outputs and small gradients. Causality requires a mask. For sequence length n, computing the score matrix requires $O(n^2 d_k)$ arithmetic.

Question 9

In a standard BART/T5 encoder-decoder model, which representations supply the queries, keys, and values for a decoder block's encoder-decoder cross-attention module? (Learned projections are applied to these source representations.)

A. Queries come from the final encoder outputs; keys and values come from the decoder hidden states.

B. Queries, keys, and values all come from the decoder hidden states.

C. Queries come from the decoder hidden states; keys and values come directly from the encoder input embeddings, bypassing the encoder layers.

D. Queries come from the decoder hidden states; keys and values come from the final encoder outputs.

Answer: D *Difficulty: Medium; Topic: Encoder-decoder cross-attention (BART/T5)*

Cross-attention lets each decoder position retrieve information from the encoded source sequence. Its queries are projections of the decoder representations at that stage of the block, while its keys and values are projections of the final encoder outputs. Using only decoder states describes self-attention; using encoder input embeddings would bypass the encoded source representations.

Question 10

Which statement about model applicability is correct?

A. T5 can perform generation tasks but cannot perform text classification by generating a class label.

B. A standard BERT encoder with its masked-language-model objective is not a left-to-right autoregressive language model.

C. A fixed-window model that uses only the mean of token embeddings, without positional information, can distinguish contexts that differ only by a permutation of the same tokens.

D. Pretrained ELMo representations alone include a task-specific text-classification head and a complete sequence-generation system.

Answer: B *Difficulty: Medium; Topic: Model applicability (T5 / BERT / ELMo)*

BERT's standard bidirectional masked-language-model setup differs from left-to-right autoregressive generation. This does not imply that no generation procedure could ever be built using BERT. T5 can express classification as generating a label. Mean pooling without positional information is invariant to token permutation. ELMo supplies contextual representations; downstream classification or generation requires additional task-specific components.

Question 11

The Python function below should compute token-level perplexity. Its input is a non-empty list containing the probability assigned to each observed next token, with $0 < p_i \leq 1$. Use natural logarithms:

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\ln p_i\right).$$

```
import math

def perplexity(token_probabilities):
    n = len(token_probabilities)
    log_sum = sum(math.log(p) for p in token_probabilities)
    return math.exp(-log_sum)  # BUG: replace this line

probabilities = [0.5, 0.25, 0.125]
print(perplexity(probabilities))
```

Which replacement for the marked return line computes perplexity correctly for all valid inputs?

A. `return math.exp(-log_sum / n)`

B. `return math.exp(log_sum / n)`

C. `return math.exp(-log_sum) / n`

D. `return -log_sum / n`

Answer: A *Difficulty: Medium; Topic: Code debugging / perplexity*

The negative log probabilities must be averaged *before* exponentiation, so A implements the supplied formula. For the three probabilities shown, the correct result is $\exp(\ln 64/3) = 4$. The original buggy function returns 64. B returns the geometric mean probability (1/4 here); C divides by N after exponentiation (64/3 here); D returns the average negative log-likelihood ($\ln 4$), rather than perplexity.

Question 12

The following PyTorch code should produce one vector of three class logits for each of two sequences. Each sequence contains five valid tokens; there is no padding. The embedding dimension is eight.
`nn.Embedding` appends the embedding dimension to the input shape. With its default `keepdim=False`, `mean(dim=...)` removes the averaged dimension. A linear layer maps the last dimension from its input size to its output size.

```
import torch
from torch import nn

token_ids = torch.tensor([[1, 2, 3, 4, 5],
                          [5, 4, 3, 2, 1]])
embedding = nn.Embedding(10, 8)
classifier = nn.Linear(8, 3)

x = embedding(token_ids)
pooled = x.mean(dim=__I__)
logits = classifier(pooled)
```

Which option correctly replaces `__I__` and states the resulting shape of `logits`?

A. `__I__` = 0; `logits.shape` is `(2, 3)`.

B. `__I__` = 1; `logits.shape` is `(2, 5, 3)`.

C. `__I__` = 1; `logits.shape` is `(2, 3)`.

D. `__I__` = 2; `logits.shape` is `(2, 3)`.

Answer: C *Difficulty: Medium; Topic: Code completion / mean pooling and tensor shapes*

Embedding lookup gives `x.shape = (2, 5, 8)`. Averaging dimension 1 pools over the five token positions separately for each sequence, giving `(2, 8)`. The linear layer maps eight features to three logits, giving `(2, 3)`. Averaging dimension 0 instead mixes the two sequences and gives logits of shape `(5, 3)`, so A is wrong. B incorrectly keeps the token dimension after pooling. Averaging dimension 2 gives `(2, 5)`, which cannot be passed to a linear layer expecting eight input features, so D raises a shape error.

Question 13

A decoder has three token IDs: 0 is the end-of-sequence token (`EOS`), while 1 and 2 are ordinary tokens. The initial prompt is the list `[1]`. The rows of `step_logits` give the next-token scores available at successive generation steps. Greedy decoding chooses the token ID with the largest score; there are no ties.

```
EOS = 0
step_logits = [[0.2, 0.1, 2.0],
               [3.0, 1.0, 0.0],
               [0.0, 4.0, 1.0]]
generated = [1]

for logits in step_logits:
    token_id = max(range(len(logits)), key=lambda i: logits[i])
    generated.append(token_id)
    if token_id == EOS:
        break

print(generated)
```

What does this code print?

A. `[2, 0]`

B. `[1, 2, 0]`

C. `[1, 2]`

D. `[1, 2, 0, 1]`

Answer: B *Difficulty: Easy; Topic: Code interpretation / greedy decoding and EOS*

The first row has its maximum at token ID 2, which is appended to the existing prompt, producing `[1, 2]`. The next row selects token ID 0. The code appends this EOS token *before* testing the stopping condition, so the final list is `[1, 2, 0]`. The third row is never processed. A drops the prompt; C drops the EOS token even though it is appended; D ignores the `break`.

CORRECTED

Question 14

The following PyTorch loop should make one ordinary optimizer update per minibatch, using only that minibatch's gradients. Gradient accumulation across minibatches is *not* intended. The model, optimizer, scalar loss function, and a sequence of at least two minibatches have already been defined.
Recall that `backward()` adds to stored parameter gradients, `zero_grad()` clears them, and `step()` updates parameters without clearing their gradients.

```
model.train()
optimizer.zero_grad()

for inputs, targets in batches:
    logits = model(inputs)
    loss = criterion(logits, targets)
    loss.backward()
    optimizer.step()
```

Which single change corrects the unintended gradient accumulation?

A. Move `optimizer.zero_grad()` inside the loop, immediately after `loss.backward()` and before `optimizer.step()`.

B. Swap `loss.backward()` and `optimizer.step()`.

C. Replace `loss.backward()` with `model.eval()`.

D. Move `optimizer.zero_grad()` to the start of each loop iteration, before the forward pass.

Answer: D *Difficulty: Medium; Topic: Code debugging / minibatch training and gradients*

The original code clears gradients only once. From the second minibatch onward, its backward pass adds new gradients to those left by previous minibatches. Clearing gradients at the start of each iteration ensures that each update uses only the current minibatch. A discards the newly computed gradients before the optimizer can use them. B steps before computing the current gradients and does not fix accumulation. C changes the model's training/evaluation mode; it neither computes nor clears the required gradients.

Corrected

Corrected

Mock Exam 2026

First name and last name:

.....

Student ID:

.....

Answer Key

Correct answers are marked.

Answers must be given exclusively on this sheet.
Answers given on the question sheets will be ignored.

Use a black or dark blue ballpen. Cross a box or fill it completely; do not tick. Leave any erased box completely white.

Question 1:	☐ **A**	☐ **B**	☐ **C**	■ **D**
Question 2:	☐ **A**	☐ **B**	☐ **C**	■ **D**
Question 3:	☐ **A**	☐ **B**	☐ **C**	■ **D**
Question 4:	■ **A**	☐ **B**	☐ **C**	☐ **D**
Question 5:	■ **A**	☐ **B**	☐ **C**	☐ **D**
Question 6:	☐ **A**	☐ **B**	☐ **C**	■ **D**
Question 7:	☐ **A**	■ **B**	☐ **C**	☐ **D**
Question 8:	☐ **A**	☐ **B**	☐ **C**	■ **D**
Question 9:	☐ **A**	☐ **B**	☐ **C**	■ **D**
Question 10:	☐ **A**	■ **B**	☐ **C**	☐ **D**
Question 11:	■ **A**	☐ **B**	☐ **C**	☐ **D**
Question 12:	☐ **A**	☐ **B**	■ **C**	☐ **D**
Question 13:	☐ **A**	■ **B**	☐ **C**	☐ **D**
Question 14:	☐ **A**	☐ **B**	☐ **C**	■ **D**

Total: 14 questions, 14 points.

Before finishing, check that you have transferred your answers to this sheet.